

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

КУРСОВАЯ РАБОТА (ПРОЕКТ)
ЗАЩИЩЕНА С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

ПРИЛОЖЕНИЕ ДЛЯ СОЗДАНИЯ И УПРАВЛЕНИЯ АВТОМОБИЛЕМ

по дисциплине: ОСНОВЫ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1	Введение.....	2
2	Анализ поставленной задачи	3
3	Диаграмма классов.....	5
3.1	Отношения в приложении	6
4	Описание программного кода классов.....	13
4.1	Класс Car	13
4.2	Класс CarPark	14
4.3	Класс Engine	15
4.4	Класс Wheel.....	16
4.5	Класс Vehicle и интерфейс IVehicle	17
4.6	Класс Component и интерфейс IComponent	18
4.7	Дополнительный класс Info.....	19
5	Тестирование функциональности классов	21
5.1	Создание и удаление автомобиля.....	21
5.2	Обычный и подробный вывод информации об автомобиле.....	22
5.3	Вывод действий пользователя.....	23
5.4	Передвижение и заправка автомобиля	24
5.5	Запуск и выключение двигателя	25
6	Использование библиотеки в приложении	27
7	Руководство пользователя	30
8	Заключение	31
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	32
	ПРИЛОЖЕНИЕ	33

1 Введение

Цель курсового проектирования как учебной дисциплины: использование полученных знаний и навыков объектно-ориентированного программирования для разработки пользовательской библиотеки классов, предназначенной для программирования приложений пользователя в конкретной предметной области. Библиотека должна учитывать особенности области приложения и располагать необходимой функциональностью для построения интерфейса пользователя, структурирования, записи, чтения данных в различных форматах, обработки, вычисления и визуализации данных. Курсовая работа позволяет:

1) систематизировать теоретические знания и закрепить полученные практические умения по дисциплине «Основы программирования» в соответствии с требованиями к уровню подготовки по направлению 09.03.02 «Информационные системы и технологии»;

2) сформировать умения работы с учебной литературой и иными информационными источниками;

3) развить профессиональную письменную и устную речь;

4) развить системное мышление, творческую инициативу, самостоятельность, организованность и ответственность за принимаемые решения;

5) сформировать навыки планомерной регулярной работы над решением поставленных задач.

2 Анализ поставленной задачи

Вариант задания: В приложении пользователь может создать объект класса Автомобиль, используя классы Колесо, Двигатель. Методы классов должны фиксировать следующие состояния: движение, заправка, замена колеса. Информация об автомобиле должна включать все технические особенности.

Предусмотреть свойства для получения состояния объекта. Описать класс авто, а также автопарк, содержащий массив автомобилей. Обеспечить следующие возможности: вывод информации автомобиля по выбранному имени; вывод информации о технических характеристиках, мощности двигателя, состояния деталей, расхода топлива, вместимости топливного бака и актуальному пробегу.

От архитектуры классов требуется:

- 1) возможность условно-постоянного хранения данных об автомобиле,
- 2) возможность хранения экземпляров автомобиля в коллекцию с множеством характеристик,
- 3) возможность изменения коллекции путем создания и удаления экземпляров,
- 4) возможность использования экземпляров для следующих задач: движение, заправка, замена колес.

От программного кода библиотеки требуется:

- 1) реализация наследования,
- 2) реализация интерфейсов,
- 3) реализация ассоциации, композиции, агрегации,
- 4) реализация полиморфизма,
- 5) реализация перегрузки и переопределения методов,
- 6) использование абстрактного класса,
- 7) программный код должен содержать минимум 4 класса и 1 интерфейс.

Также требуется на разработать понятный интерфейс клиента: подписать поля ввода, элементы управления, показ информации объектов, состояния объектов.

Полученная библиотека должна использоваться для тестирования автомобилей с разными характеристиками с помощью различных методов взаимодействия с объектом. А также иметь возможность загрузки и выгрузки моделей автомобилей из автопарка для дальнейшей работы.

3 Диаграмма классов

На рисунке 1 представлена диаграмма классов разработанного приложения.

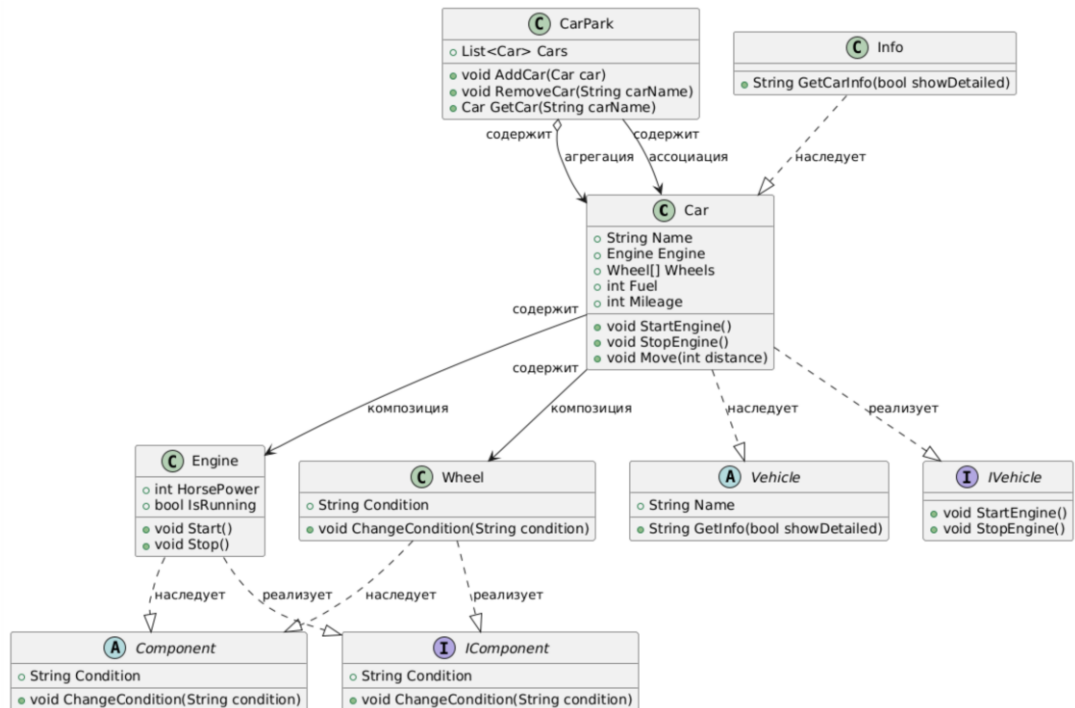


Рисунок 1 – Диаграмма классов приложения

На рисунке видно, что:

- 1) класс Car ассоциирован с классами Wheel, Engine посредством композиции. Объекты классов Engine и Wheel являются составными частями объекта Car,
- 2) класс Car используется интерфейс IComponent, который определяет базовые методы взаимодействия с компонентами автомобиля, такие как ChangeCondition. Это реализовано в классах Wheel и Engine,
- 3) класс Engine реализует интерфейс IComponent. Он включает дополнительные свойства и методы, такие как HorsePower, isRunning, и методы Start, Stop, ChangeCondition,

4) класс `Wheel` также реализует интерфейс `IComponent`. Помимо методов интерфейса, он имеет дополнительные свойства и методы, такие как `BrokenCount`, `Repair`, которые позволяют работать с состоянием колеса,

5) абстрактный класс `Vehicle` содержит базовые свойства и методы, такие как `Name` и метод `GetInfo`, который реализует перегрузку в классе `Info`. От этого класс `Vehicle` наследуется класс `Car`, который расширяет функциональность базового класса,

6) абстрактный класс `Component` используется как общий базовый класс для компонентов автомобиля (`Engine` и `Wheel`). Он включает базовые свойства `Condition` и метод `ChangeCondition`, которые переопределяются в дочерних классах,

7) класс `Car` взаимодействует с топливом и движением через методы `Move` и `Refuel`, реализуя полиморфизм в рамках функциональности наследования от `Vehicle`,

8) класс `CarPark` представляет собой коллекцию автомобилей. Он содержит методы для добавления, удаления и поиска автомобилей по имени. Это реализует возможность хранения и управления экземплярами класса `Car`,

9) класс `CarPark` ассоциирован с классом `Car` посредством агрегации, так как экземпляры автомобилей добавляются в массив внутри `CarPark`,

10) в классе `Car` методы `StartEngine` и `StopEngine` переопределяют аналогичные методы из базового класса `Vehicle`,

11) метод `ChangeCondition` реализован полиморфно как в классе `Engine`, так и в классе `Wheel`, адаптируясь под специфику компонента.

3.1 Отношения в приложении

Связь между классами `CarPark` и `Car` представляет собой ассоциацию. В данной связи класс `CarPark` содержит список автомобилей, что позволяет управлять коллекцией объектов `Car`. Эта ассоциация демонстрирует, что `CarPark` может добавлять или удалять автомобили из своего списка, однако сами автомобили могут существовать независимо от автопарка. Таким

образом, CarPark и Car имеют взаимосвязь, но не зависят друг от друга в плане существования. На рисунке 2 представлена UML ассоциации класса Car и CarPark.

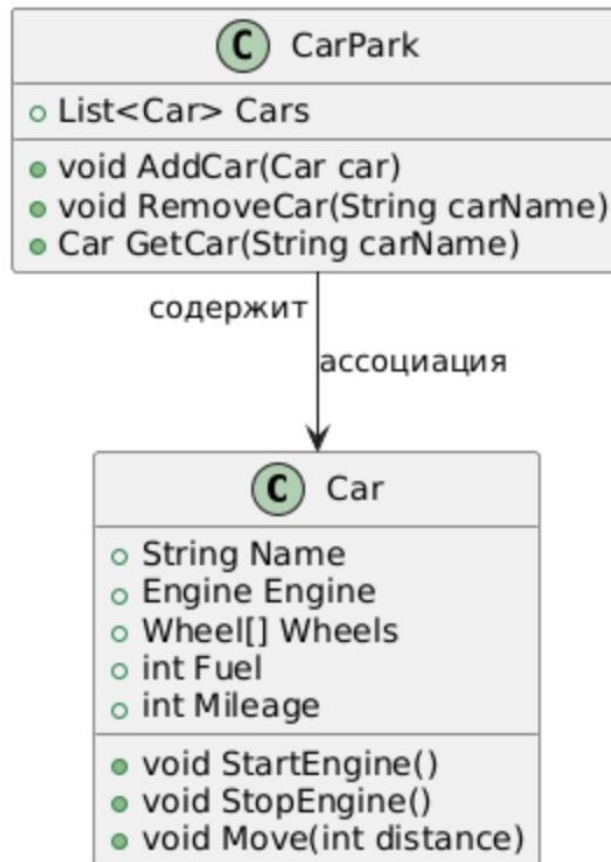


Рисунок 2 – UML ассоциации класса Car и CarPark

Связь между классом Car и классами Engine и Wheel представляет собой композицию. В данной связи класс Car содержит объекты Engine и массив Wheel, что означает, что эти компоненты являются неотъемлемой частью автомобиля. Если объект Car будет уничтожен, то все его компоненты, такие как Engine и Wheel, также будут уничтожены. Это подчеркивает сильную зависимость между классом Car и его компонентами, так как они не могут существовать отдельно от автомобиля. На рисунке 3 представлена UML композиции класса Car с классами Engine и Wheel.

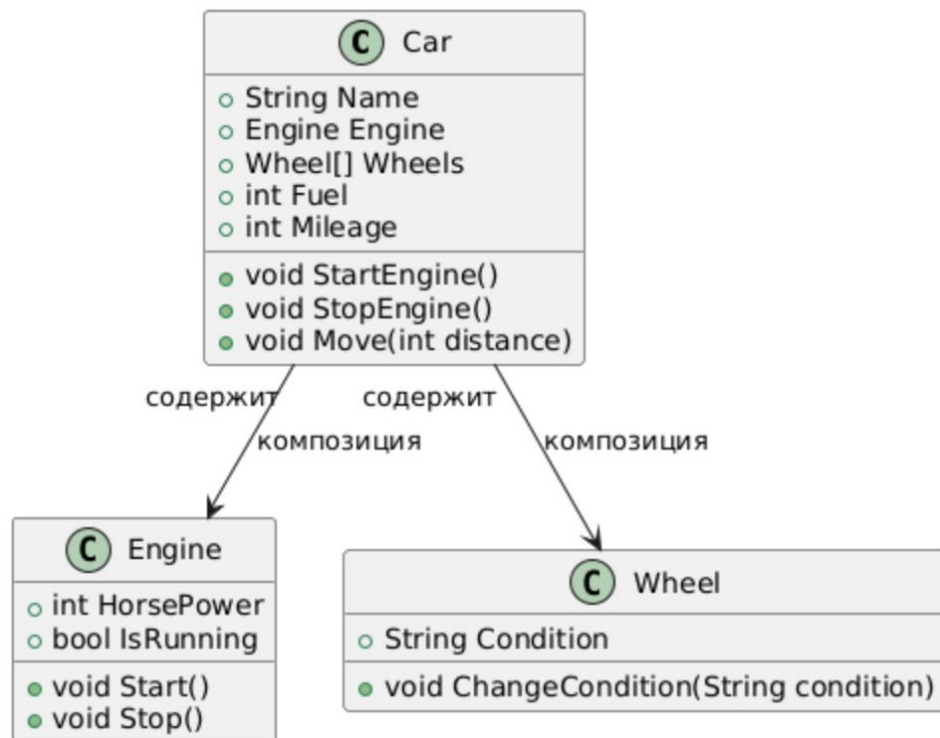


Рисунок 3 – UML композиции класса Car с классами Engine и Wheel

Связь между классом CarPark и классом Car представляет собой агрегацию. В этой связи класс CarPark содержит коллекцию объектов Car, что позволяет управлять множеством автомобилей. Однако, в отличие от композиции, объекты Car могут существовать независимо от CarPark. Это означает, что даже если CarPark будет уничтожен, автомобили, находящиеся в нем, могут продолжать существовать. Таким образом, агрегация демонстрирует более слабую зависимость между классами по сравнению с композицией. На рисунке 4 представлена UML агрегации класса CarPark с классом Car.

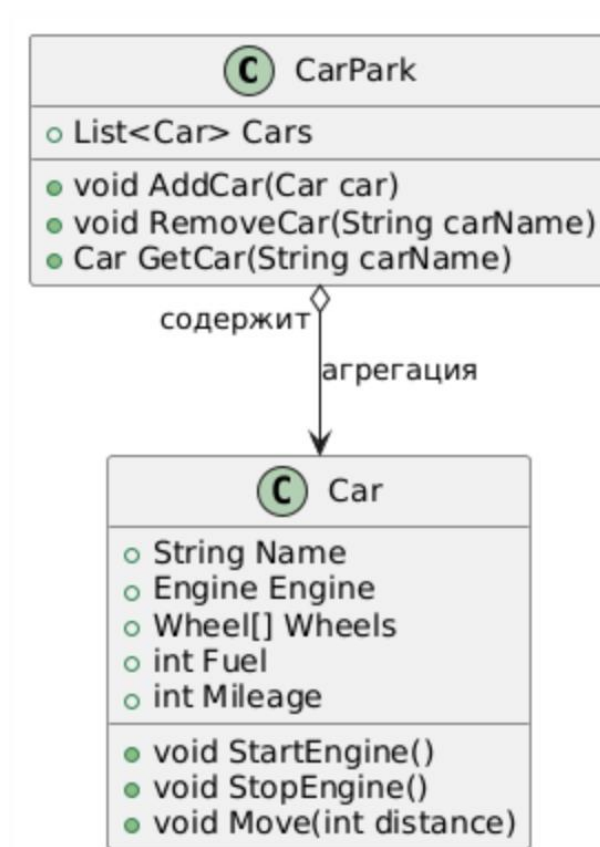


Рисунок 4 – UML агрегации класса CarPark с классом Car.

Связь между классом Car и классом Vehicle, а также между классом Info и классом Car представляет собой наследование. Класс Car наследует свойства и методы класса Vehicle, что позволяет ему использовать общие функциональные возможности, такие как получение информации о транспортном средстве. Класс Info, в свою очередь, наследует от класса Car, что позволяет ему расширять функциональность автомобиля, добавляя возможность получения детализированной информации. Это демонстрирует принцип наследования в объектно-ориентированном программировании, где дочерние классы могут использовать и расширять функциональность родительских классов. На рисунке 5 представлена UML наследования между классами Vehicle, Car и Info.

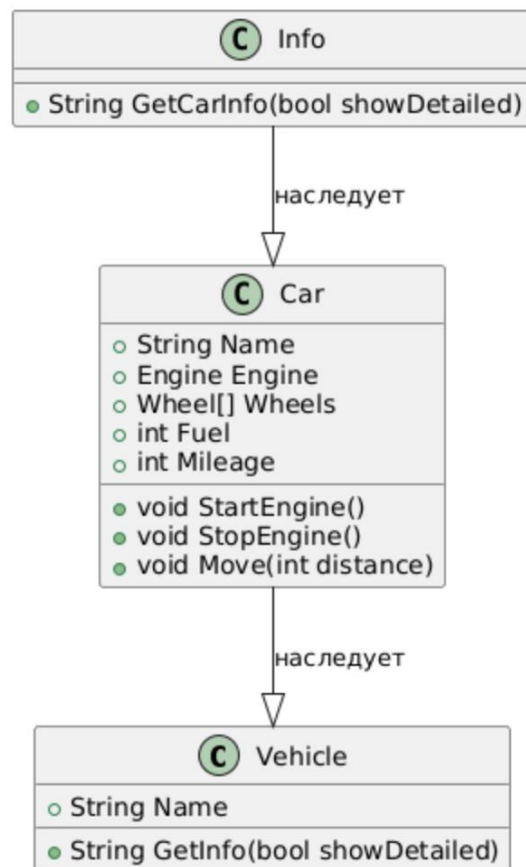


Рисунок 5 – UML наследования между классами Vehicle, Car и Info.

Связь между классами Car, Engine, Wheel и интерфейсами IVehicle и IComponent представляет собой реализацию интерфейсов. Класс Car реализует интерфейс IVehicle, что позволяет ему предоставлять методы для управления автомобилем, такие как запуск и остановка двигателя. Классы Engine и Wheel реализуют интерфейс IComponent, что позволяет им определять методы для изменения состояния компонентов, таких как двигатель и колеса. Это демонстрирует принцип полиморфизма в объектно-ориентированном программировании, где разные классы могут предоставлять свои собственные реализации методов, определенных в интерфейсах. На рисунке 6 представлена UML реализации интерфейсов для классов Car, Engine и Wheel.

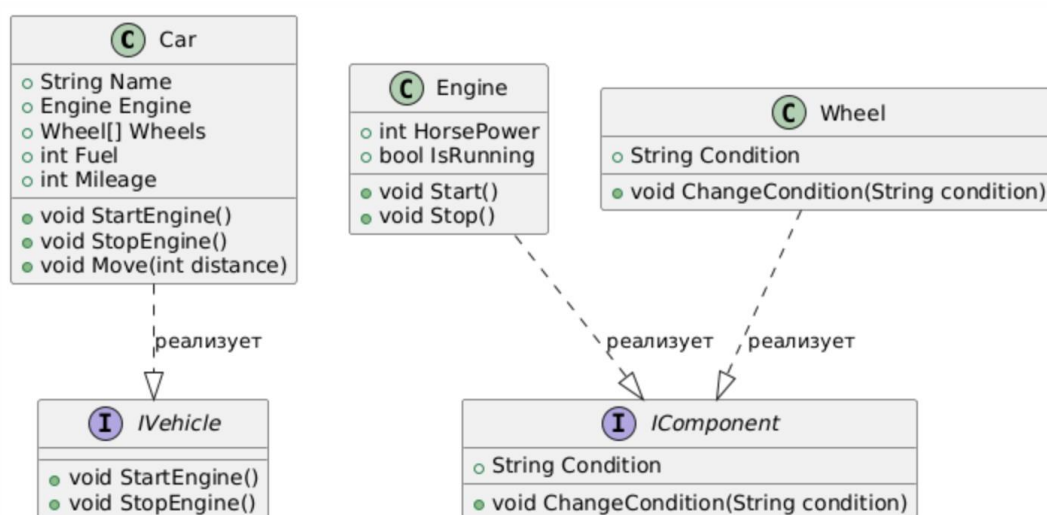


Рисунок 6 – UML реализации интерфейсов для классов Car, Engine и Wheel.

Связь между классами Car, Engine, Wheel и абстрактными классами Vehicle и Component представляет собой наследование от абстрактных классов. Класс Car наследует от абстрактного класса Vehicle, что позволяет ему использовать общие свойства и методы для всех транспортных средств. Классы Engine и Wheel наследуют от абстрактного класса Component, что позволяет им использовать общие методы и свойства для всех компонентов автомобиля. Это демонстрирует принцип наследования в объектно-ориентированном программировании, где абстрактные классы могут определять общие интерфейсы для своих дочерних классов. На рисунке 7 представлена UML наследования для абстрактных классов Vehicle и Component.

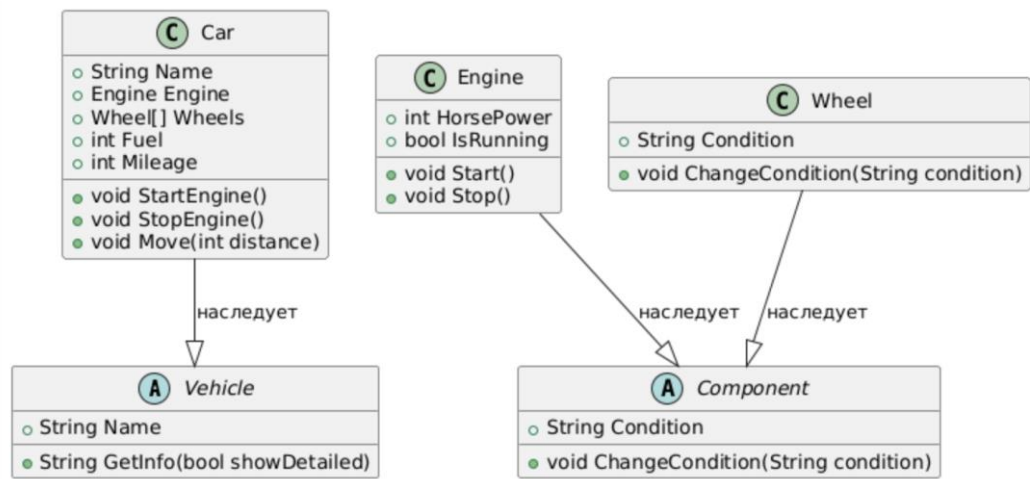


Рисунок 7 – UML наследования для абстрактных классов Vehicle и Component.

4 Описание программного кода классов

4.1 Класс Car

Класс Car представляет собой основную сущность в приложении для управления автомобилями. Он включает в себя свойства, такие как название автомобиля, объект двигателя и массив колес. Класс также управляет состоянием топлива, пробегом и расходом топлива. Важной частью класса является возможность запуска и остановки двигателя, а также перемещения автомобиля на заданное расстояние с проверкой состояния компонентов. На рисунке 2 представлен код класса.

```
using System;
using System.Linq;

namespace CarManagementApp
{
    // Класс Car представляет автомобиль с его компонентами (двигатель, колеса) и управлением ими
    // Ссылка: 1 | Code | explain | explain step by step | doc | test
    public class Car
    {
        // Свойство Name хранит название автомобиля с обработкой лишнего пробела
        // Ссылка: 2
        private string _name;
        public string Name
        {
            get { return _name; }
            set { _name = value.Trim(); }
        }

        // Свойство Engine хранит объект двигателя автомобиля
        // Ссылка: 10
        public Engine Engine { get; set; }

        // Свойство Wheels хранит массив колес автомобиля (массив из 4 колес)
        // Ссылка: 11
        public Wheel[] Wheels { get; set; }

        // Свойство Fuel хранит текущее количество топлива в автомобиле с проверкой на отрицательные значения
        // Ссылка: 2
        private int _fuel;
        // Ссылка: 8
        public int Fuel
        {
            get { return _fuel; }
            set { _fuel = value > 0 ? value : 0; }
        }

        // Свойство FuelTankCapacity хранит максимальный объем топливного бака
        // Ссылка: 5
        public int FuelTankCapacity { get; private set; }

        // Свойство Mileage хранит пробег автомобиля в километрах
        // Ссылка: 4
        public int Mileage { get; set; } = 0;

        // Свойство FuelConsumption хранит расход топлива на 100 км
        // Ссылка: 5
        public int FuelConsumption { get; private set; }

        // Статическое свойство для общего количества колес
        // Ссылка: 1
        public static int WheelCount { get; } = 4;

        // Свойство Symbol проверяет регистр символа и преобразует строчные буквы в прописные
        // Ссылка: 2
        private char _symbol;
        // Ссылка: 0
        public char Symbol
        {
            get { return _symbol; }
            set { _symbol = char.ToUpper(set); }
        }

        // Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность
        // Ссылка: 2
        private DateTime _lastServiceDate;
        // Ссылка: 0
        public DateTime LastServiceDate
        {
            get { return _lastServiceDate; }
            set { _lastServiceDate = DateTime.Parse(set); }
        }

        // Конструктор класса Car
        // Инициализирует автомобиль с заданными параметрами: имя, мощность двигателя, расход топлива и объем бака
        // Ссылка: 0
        public Car(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
        {
            Name = name;
            Engine = new Engine(horsepower);
            FuelConsumption = fuelConsumption;
            FuelTankCapacity = fuelTankCapacity;
            Wheels = new Wheel[WheelCount];
        }

        // Свойство IsEngineOn проверяет, работает ли двигатель
        // Возвращает true, если двигатель включен, иначе false
        // Ссылка: 2
        public bool IsEngineOn => Engine.IsRunning;

        // Метод StartEngine запускает двигатель
        // Ссылка: 0
        public void StartEngine() => Engine.Start();

        // Метод StopEngine выключает двигатель
        // Ссылка: 0
        public void StopEngine() => Engine.Stop();

        // Обработчик события поломки компонента (колеса или двигателя)
        // При поломке компонента выводится сообщение о поломке в консоль
        // Ссылка: 2
        private void OnComponentBroken(object sender, EventArgs e)
        {
            Console.WriteLine($"Поломка компонента: {sender}");
        }

        // Перегрузка метода Move: стандартное движение (обычная скорость)
        // Этот метод вызывает второй метод Move с параметром isFast = false (стандартная скорость)
        // Ссылка: 0
        public void Move(int distance)
        {
            Move(distance, false);
        }

        // Перегрузка метода Move: движение с дополнительными параметрами (например, высокая скорость)
        // Если параметр isFast равен true, расход топлива снижается на 20% (ускоренный режим)
        // Ссылка: 1
        public void Move(int distance, bool isFast = false)
        {
            // Расчет расхода топлива
            int fuelConsumptionPer100km = FuelConsumption * (isFast ? 0.8 : 1.0);
            int fuelConsumptionForDistance = (fuelConsumptionPer100km * distance) / 100;

            // Проверка наличия топлива
            if (Fuel < fuelConsumptionForDistance)
            {
                Console.WriteLine("Недостаточно топлива для поездки на такое расстояние.");
                return;
            }

            // Обновление топлива и пробега
            Fuel -= fuelConsumptionForDistance;
            Mileage += distance;

            // Проверка состояния колес
            foreach (var wheel in Wheels)
            {
                if (wheel.IsBroken)
                {
                    ReplaceBrokenWheel();
                }
            }
        }

        // Метод Refuel запрашивает автомобиль на указанное количество литров
        // Проверяет, не превышает ли заправка емкость бака
        // Ссылка: 0
        public void Refuel(int amount)
        {
            if (amount < 0)
            {
                Console.WriteLine("Количество топлива не может быть отрицательным.");
                return;
            }

            if (Fuel + amount > FuelTankCapacity)
            {
                Console.WriteLine("Бака не хватит для такой заправки.");
                return;
            }

            Fuel += amount;
        }

        // Метод ReplaceBrokenWheel заменяет сломанное колесо на новое
        // Ссылка: 0
        public void ReplaceBrokenWheel()
        {
            // Поиск сломанного колеса
            for (int i = 0; i < Wheels.Length; i++)
            {
                if (Wheels[i].IsBroken)
                {
                    // Замена колеса
                    Wheels[i] = new Wheel();
                    break;
                }
            }
        }

        // Метод для получения информации о машине
        // Если параметр showDetailed = true, выводится подробная информация
        // Ссылка: 0
        public virtual string GetCarInfo(bool showDetailed = false)
        {
            string info = $"Название: {Name}, Мощность: {Engine.HorsePower} л.с., Расход: {FuelConsumption} л/100км, Пробег: {Mileage} км, Топливо: {Fuel} л, Бака: {FuelTankCapacity} л, Символ: {Symbol}";

            if (showDetailed)
            {
                info += "\nДетали: Двигатель: " + Engine.ToString() + ", Колеса: ";
                foreach (var wheel in Wheels)
                {
                    info += wheel.ToString() + " ";
                }
            }

            return info;
        }
    }
}
```

Рисунок 2 – Краткий код класса Car

Конструктор класса инициализирует все необходимые параметры, создавая экземпляры двигателя и колес. Класс также обрабатывает события поломки компонентов, выводя сообщения о проблемах с автомобилем. Это позволяет пользователю оперативно реагировать на неисправности и поддерживать автомобиль в рабочем состоянии.

4.2 Класс CarPark

Класс CarPark отвечает за управление коллекцией автомобилей в приложении. Он содержит список автомобилей и предоставляет методы для добавления и удаления автомобилей из этого списка. Класс позволяет пользователю легко управлять автопарком, обеспечивая простоту взаимодействия с автомобилями. На рисунке 3 представлен код класса.

```
using System;
using System.Collections.Generic;

namespace CarManagementApp
{
    Ссылка: 0 | GigaCode: explain | explain step by step | doc | test
    public class CarPark
    {
        // Список автомобилей в автопарке
        Ссылка: 4
        public List<Car> Cars { get; set; } = new List<Car>();

        // Добавление автомобиля в автопарк
        Ссылка: 0
        public void AddCar(Car car)
        {
            Cars.Add(car); // Добавляем новый объект Car в список Cars
        }

        // Удаление автомобиля из автопарка по имени
        Ссылка: 0
        public void RemoveCar(string carName)
        {
            // Ищем машину по имени в списке
            Car carToRemove = Cars.Find(c => c.Name == carName);
            if (carToRemove != null)
            {
                Cars.Remove(carToRemove); // Удаляем найденный объект Car
            }
        }

        // Получение автомобиля по имени
        Ссылка: 0
        public Car GetCar(string carName)
        {
            // Ищем машину по имени в списке и возвращаем её
            return Cars.Find(c => c.Name == carName);
        }
    }
}
```

Рисунок 3 – Код класса CarPark

Методы класса включают добавление нового автомобиля в список, удаление автомобиля по имени и получение автомобиля по имени. Это позволяет пользователю эффективно управлять своим автопарком, а также поддерживать актуальную информацию о доступных автомобилях. Класс CarPark является важным компонентом системы, обеспечивая централизованное управление всеми автомобилями.

4.3 Класс Engine

Класс Engine представляет собой компонент автомобиля, отвечающий за его двигатель. Он содержит свойства, такие как мощность двигателя и состояние, которое может быть "включен" или "выключен". Класс управляет работой двигателя, позволяя запускать и останавливать его, а также обрабатывать события поломки. На рисунке 4 представлен код класса.

```
using System;

// Класс Engine наследуется от Component и представляет двигатель автомобиля
// Класс управляет состоянием двигателя (работает/не работает) и его мощностью
Ссылка: 0 | GigaCode: explain | explain step by step | doc | test
public class Engine : Component
{
    // Свойство HorsePower хранит мощность двигателя в лошадиных силах
    Ссылка: 0
    public int HorsePower { get; set; }

    // Свойство IsRunning указывает, работает ли двигатель (true - работает, false - не работает)
    Ссылка: 2
    public bool IsRunning { get; private set; }

    // Метод Start запускает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    Ссылка: 0
    public void Start()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя запустить.");
        IsRunning = true;
    }

    // Метод Stop останавливает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    Ссылка: 0
    public void Stop()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя остановить.");
        IsRunning = false;
    }

    // Переопределение метода ChangeCondition для изменения состояния двигателя
    // Изменяет состояние и вызывает событие поломки, если двигатель стал "Сломан"
    Ссылка: 0
    public void ChangeCondition(string condition)
    {
        Condition = condition;
    }
}
```

Рисунок 4 – Код класса Engine

Методы класса включают запуск и остановку двигателя, а также обработку событий, связанных с его состоянием. При поломке двигателя класс генерирует событие, которое может быть обработано другими компонентами системы, например, классом Car. Это позволяет пользователю получать информацию о состоянии двигателя и принимать необходимые меры для его обслуживания.

4.4 Класс Wheel

Класс Wheel представляет собой компонент автомобиля, отвечающий за колеса. Он содержит свойства, такие как состояние колеса, которое может быть "новое" или "сломано", а также счетчик поломок. Класс управляет состоянием колеса и предоставляет методы для его восстановления и изменения состояния. На рисунке 5 представлен код класса.

```
using System;

// Ссылка: 0 | OrgaCode: explain | explain step by step | doc | test
public class Wheel : IComponent
{
    // Ссылка: 5
    public string Condition { get; set; } = "Новое"; // Состояние колеса
    // Ссылка: 2
    public int BrokenCount { get; set; } = 0; // Счетчик поломок колеса

    // Событие поломки колеса
    // Ссылка: 2
    public event EventHandler ComponentBroken;

    // Метод для восстановления колеса
    // Ссылка: 2
    public void Repair()
    {
        // Если колесо уже новое, невозможно выполнить восстановление
        if (Condition == "Новое")
        {
            throw new InvalidOperationException("Колесо уже в хорошем состоянии.");
        }
        // Восстановление состояния колеса в "Новое"
        Condition = "Новое";
        // Генерация события о восстановлении колеса
        ComponentBroken?.Invoke(this, EventArgs.Empty);
    }

    // Метод для изменения состояния колеса
    // Ссылка: 0
    public void ChangeCondition(string condition)
    {
        // Проверка, что состояние колеса либо "Новое", либо "Сломано"
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        // Изменение состояния колеса
        Condition = condition;

        // Если колесо сломано, увеличиваем счетчик поломок и генерируем событие
        if (Condition == "Сломано")
        {
            BrokenCount++; // Увеличиваем счетчик поломок
            ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события при поломке
        }
    }

    // Метод для получения информации о колесе
    // Ссылка: 0
    public string GetInfo(bool showDetailed)
    {
        // Базовая информация о состоянии колеса
        string info = $"Колесо состояние: {Condition}";

        if (showDetailed)
        {
            // Если запрашивается подробность, добавляем информацию о количестве поломок
            info += $"\nКоличество поломок: {BrokenCount}";
        }

        return info; // Возвращаем итоговую информацию
    }
}
```

Рисунок 5 – Код класса Wheel

Методы класса включают изменение состояния колеса и восстановление его в "новое" состояние. При поломке колеса класс генерирует событие,

которое может быть обработано другими компонентами системы. Это позволяет пользователю отслеживать состояние колес и принимать меры для их обслуживания, что является важным аспектом управления автомобилем.

4.5 Класс Vehicle и интерфейс IVehicle

Класс Vehicle является абстрактным базовым классом для всех транспортных средств в приложении. Он определяет общие свойства, такие как название транспортного средства, и методы для получения информации о нем. Класс включает виртуальный метод GetInfo, который возвращает основную информацию о транспортном средстве, а также защищенный метод GetDetailedInfo для предоставления более подробной информации в производных классах. На рисунке 6 представлен код класса.

```
using System;

Ссылка: 0 | GigaCode: explain | explain step by step | doc | test
public abstract class Vehicle
{
    Ссылка: 2
    public string Name { get; set; } // Название транспортного средства

    // Базовый метод для получения информации о транспортном средстве.
    // Этот метод будет переопределяться в наследующих классах.
    Ссылка: 0
    public virtual string GetInfo(bool showDetailed)
    {
        // Строка с основной информацией о транспортном средстве
        string info = $"Название: {Name}";

        if (showDetailed)
        {
            // Добавление подробной информации, если параметр showDetailed равен true
            info += GetDetailedInfo();
        }

        return info; // Возвращаем информацию о транспортном средстве
    }

    // Этот метод можно переопределить в дочерних классах для предоставления подробной информации
    Ссылка: 1
    protected virtual string GetDetailedInfo()
    {
        // Возвращаем информацию о названии транспортного средства
        return $"Название: {Name}";
    }
}
```

Рисунок 6 – Код класса Vehicle

Интерфейс IVehicle определяет контракт для управления транспортными средствами. Он включает методы для запуска и остановки двигателя, что позволяет реализовать единый подход к управлению

различными типами транспортных средств. Класс `Vehicle` и интерфейс `IVehicle` работают вместе, обеспечивая гибкость и расширяемость системы, позволяя создавать различные типы транспортных средств с общими функциональными возможностями. На рисунке 6 представлен код класса.

```
namespace CarManagementApp
{
    // Интерфейс IVehicle, который определяет методы для управления транспортным средством
    // Ссылка: 0
    public interface IVehicle
    {
        // Метод для запуска двигателя
        // Ссылка: 0
        void StartEngine(); // Запуск двигателя

        // Метод для остановки двигателя
        // Ссылка: 0
        void StopEngine(); // Остановка двигателя
    }
}
```

Рисунок 7 – Код интерфейса `IVehicle`

4.6 Класс `Component` и интерфейс `IComponent`

Класс `Component` является абстрактным базовым классом для всех компонентов автомобиля, таких как двигатель и колеса. Он определяет общие свойства, такие как состояние компонента, и методы для изменения этого состояния. Класс также включает механизм обработки событий, позволяющий уведомлять о поломках компонентов, что важно для управления состоянием автомобиля. На рисунке 8 представлен код класса.

```
using System;

// Ссылка: 0 | GigaCode: explain | explain step by step | doc | test
public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    // Ссылка: 2
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    // Ссылка: 0
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защищенный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    // Ссылка: 1
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    // Ссылка: 1
    public event EventHandler ComponentBroken;
}
```

Рисунок 7 – Код класс Component

Интерфейс `IComponent` определяет контракт для компонентов автомобиля, включая метод `GetInfo`, который возвращает информацию о состоянии компонента. Это позволяет обеспечить единообразие в реализации различных компонентов и упрощает взаимодействие с ними. Класс `Component` и интерфейс `IComponent` работают вместе, обеспечивая гибкость и возможность расширения системы, позволяя добавлять новые компоненты с минимальными изменениями в коде. На рисунке 9 представлен код класса.

```
using System;

Ссылка: 0
public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    Ссылка: 0
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    Ссылка: 0
    void ChangeCondition(string condition);
}
```

Рисунок 9 – Код интерфейса IComponent

4.7 Дополнительный класс Info

Класс `Info` наследуется от класса `Car` и предназначен для предоставления информации о конкретном автомобиле. Он расширяет функциональность базового класса `Car`, добавляя возможность получения детализированной информации о состоянии автомобиля, включая его пробег, состояние двигателя и колес, а также расход топлива. На рисунке 10 представлен код класса.

```

using System.Linq;

namespace CarManagementApp
{
    Ссылка: 1 | GigaCode: explain | explain step by step | doc | test
    public sealed class Info : Car
    {
        // Конструктор для класса Info, который вызывает конструктор базового класса
        Ссылка: 0
        public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
            : base(name, horsepower, fuelConsumption, fuelTankCapacity)
        {
        }

        // Переопределяем метод получения информации о машине
        Ссылка: 0
        public override string GetCarInfo(bool showDetailed = false)
        {
            string info = $"Название: {Name}\n" +
                $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
                $"Пробег: {Mileage} км\n" +
                $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
                $"Расход топлива: {FuelConsumption} л/100 км\n" +
                $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
                $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

            if (showDetailed)
            {
                info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
                for (int i = 0; i < Wheels.Length; i++)
                {
                    info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
                }
            }

            return info;
        }
    }
}

```

Рисунок 10 – Код класса Info

Метод `GetCarInfo` переопределен в классе `Info`, что позволяет возвращать как основную, так и подробную информацию о машине в зависимости от переданного параметра. Если параметр `showDetailed` установлен в `true`, метод предоставляет дополнительную информацию, такую как оставшийся пробег до поломки двигателя и состояние каждого колеса.

Класс `Info` обеспечивает более глубокое понимание состояния автомобиля, что позволяет пользователю принимать обоснованные решения о его обслуживании и эксплуатации.

5 Тестирование функциональности классов

Для реализации тестирования функциональности библиотеки был создан графический интерфейс с возможностями:

- 1) создания и удаления автомобиля,
- 2) обычный и подробный показ информации экземпляра выбранного автомобиля,
- 3) вывод информации о последних действиях пользователя,
- 4) передвижение, заправка автомобиля. Также реализована возможность езды с высокой скоростью, например на шоссе,
- 5) запуск и выключение двигателя,
- 6) замена автомобильного колеса.

На рисунке 11 представлен интерфейс приложения.

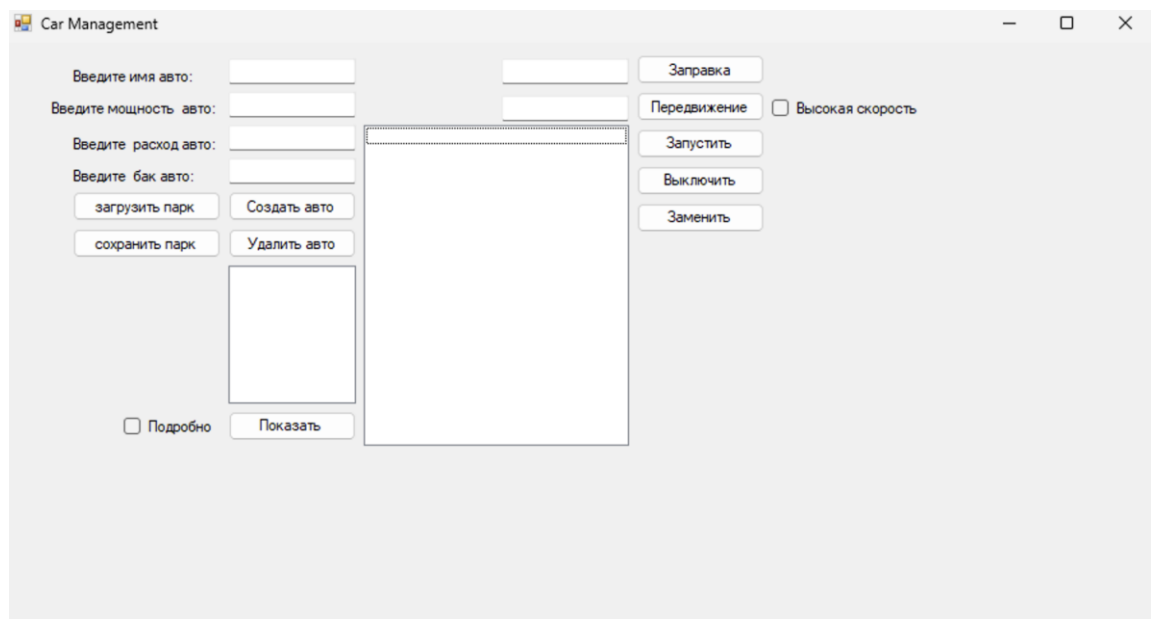


Рисунок 11 – Интерфейс приложения

5.1 Создание и удаление автомобиля

В первую очередь протестируем добавление и удалению экземпляров авто в нашем интерфейсе. Создадим автомобиль Lada, удалим его, и создадим еще один автомобиль Lada Pro. На рисунке 12 представлен скриншот теста добавления и удаления автомобиля.

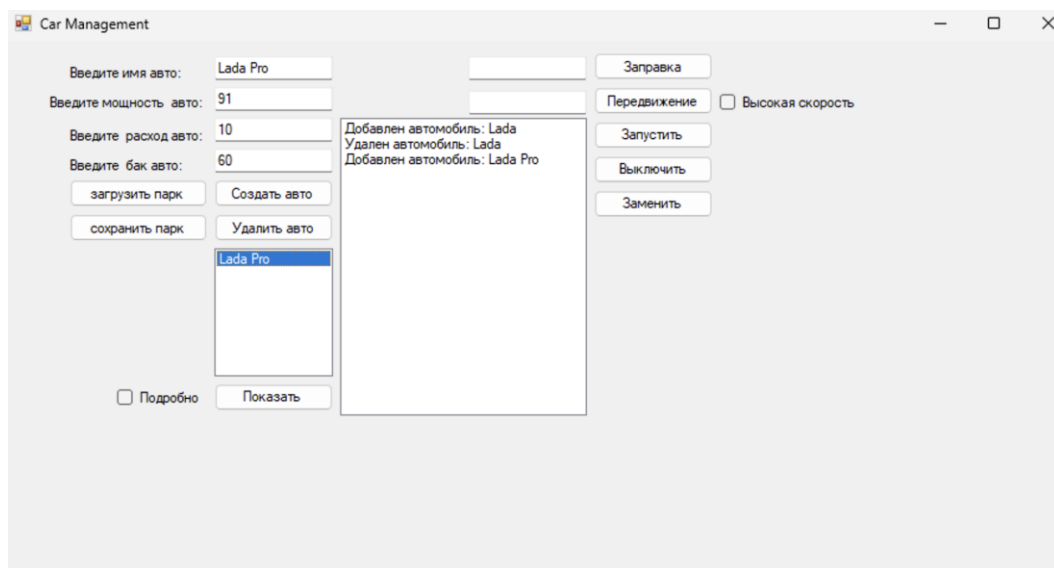


Рисунок 12 – Тест добавления и удаления автомобиля

5.2 Обычный и подробный вывод информации об автомобиле

Протестируем обычный и подробный показ информации автомобиля в интерфейсе. Изучим автомобиль Lada, информация должна иметь все введенные данные при создании экземпляра. При подробном просмотре выводится состояния компонентов двигателя и четырех колес автомобиля. На рисунке 13 продемонстрирован тест обычного вывода информации об автомобиле.

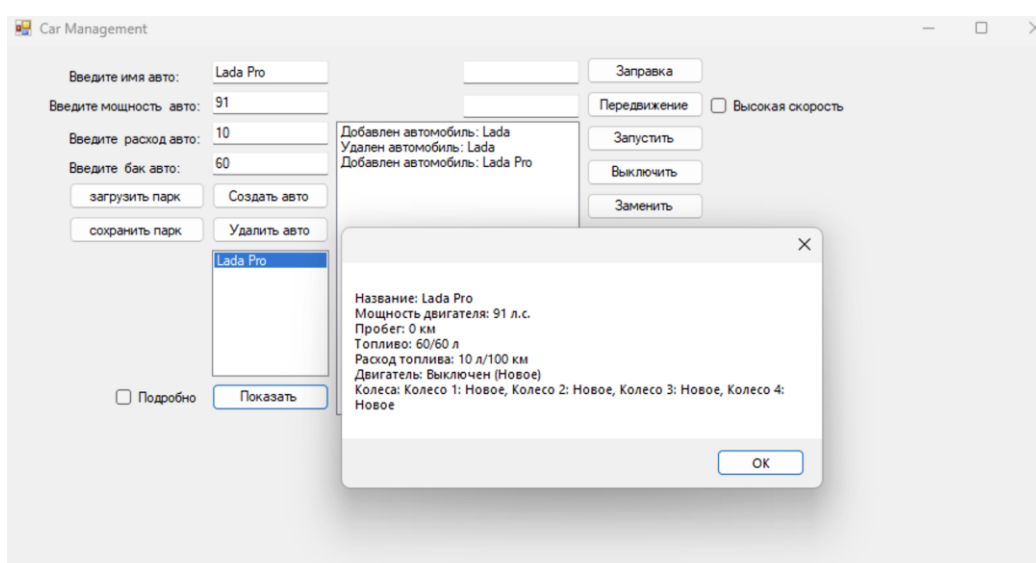


Рисунок 13 – Тест обычного вывода информации об автомобиле

На рисунке 14 представлен тест подробного вывода информации об автомобиле.

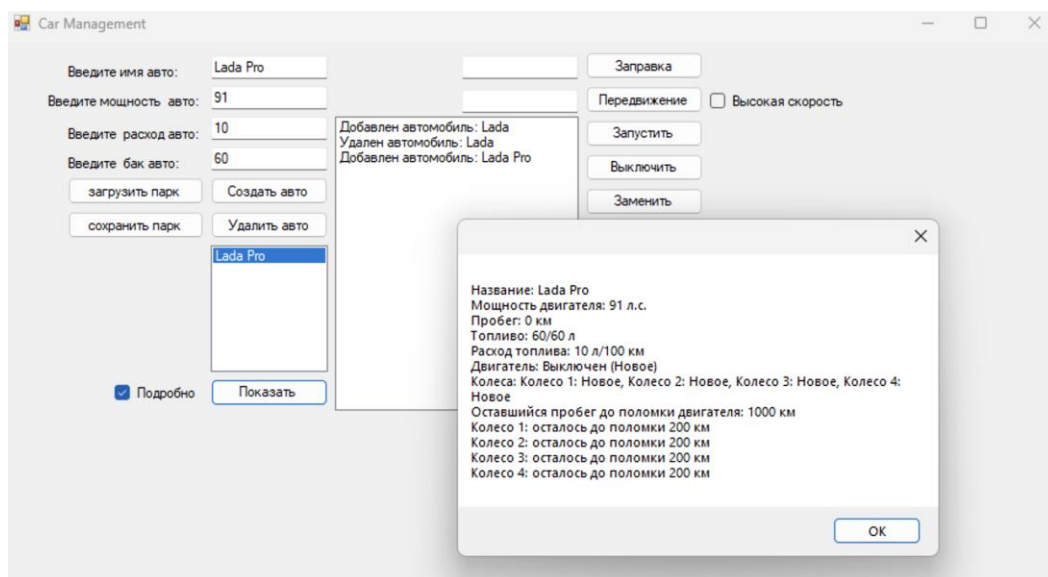


Рисунок 14 – Тест подробного вывода информации об автомобиле

5.3 Вывод действий пользователя

Протестируем вывод всех действий пользователя с данным интерфейсом. На рисунке 15 представлен тест вывода всех действий пользователя.

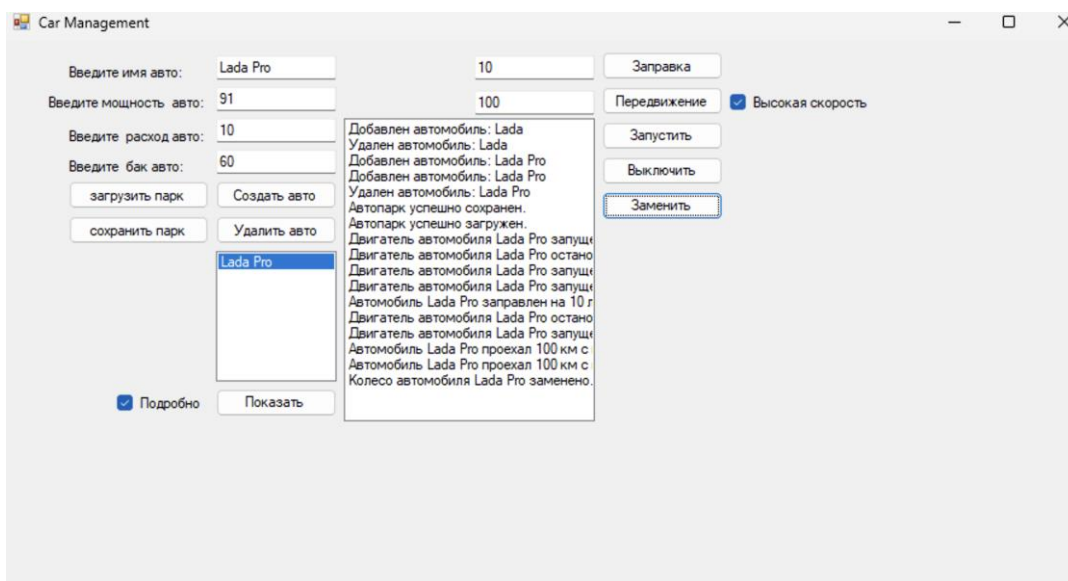


Рисунок 15 – Тест вывода всех действий пользователя

5.4 Передвижение и заправка автомобиля

Протестируем езду с обычной скоростью и высокой (на шоссе), а также заправку. На рисунке 16 представлен тест езды с обычной скоростью.

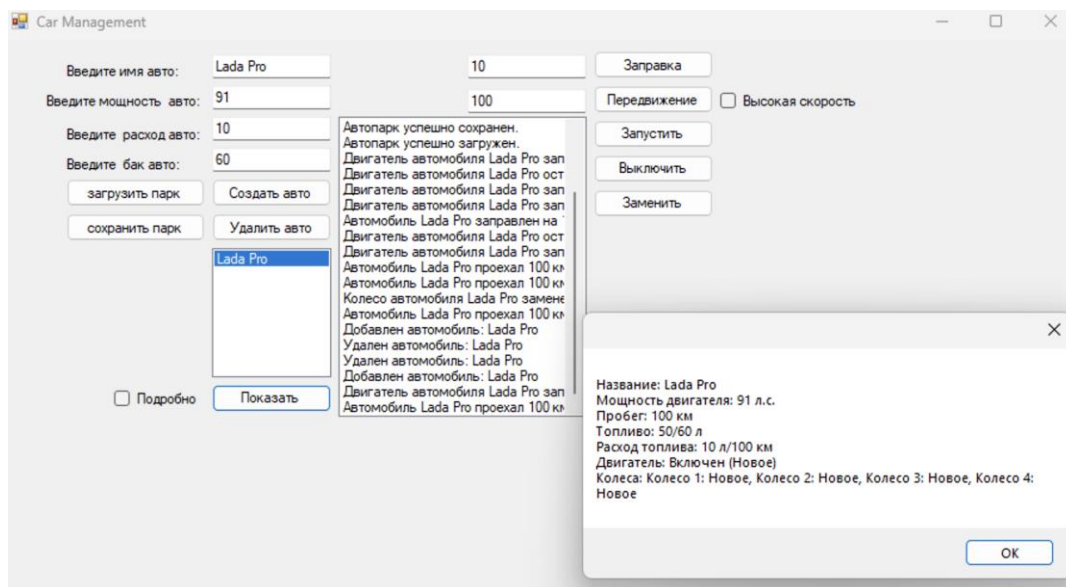


Рисунок 16 – Тест езды с обычной скоростью

Езда с обычной скоростью работает правильно, тратит то количество топлива, которое было заложено. Пробег начислен правильно. На рисунке 17 представлен тест езды с высокой скоростью.

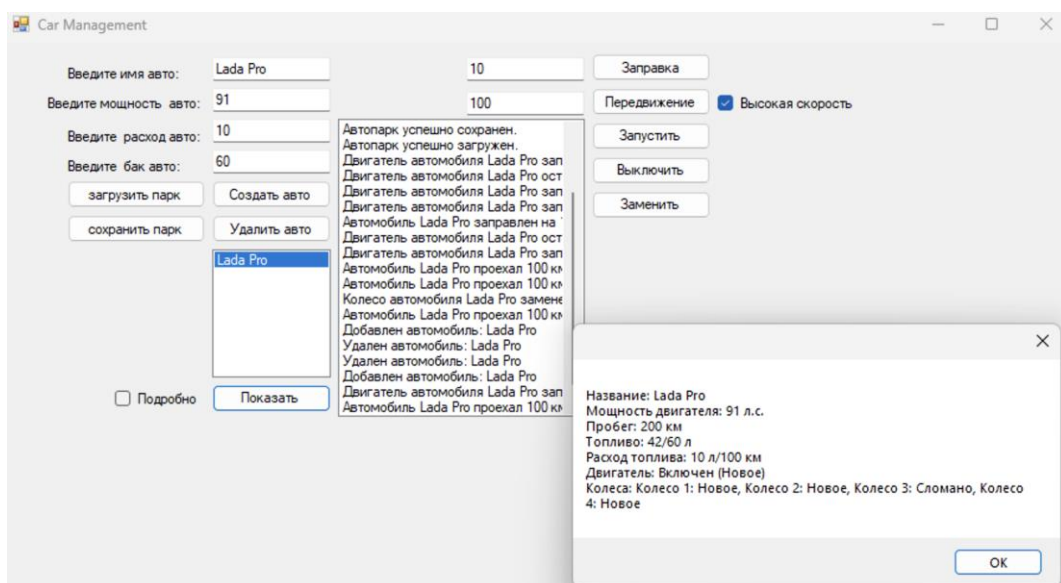


Рисунок 17 – Тест езды с высокой скоростью

Езда с высокой скоростью работает правильно, тратит 80% от изначального расхода. Пробег начислен правильно. На рисунке 18 представлен тест заправки.

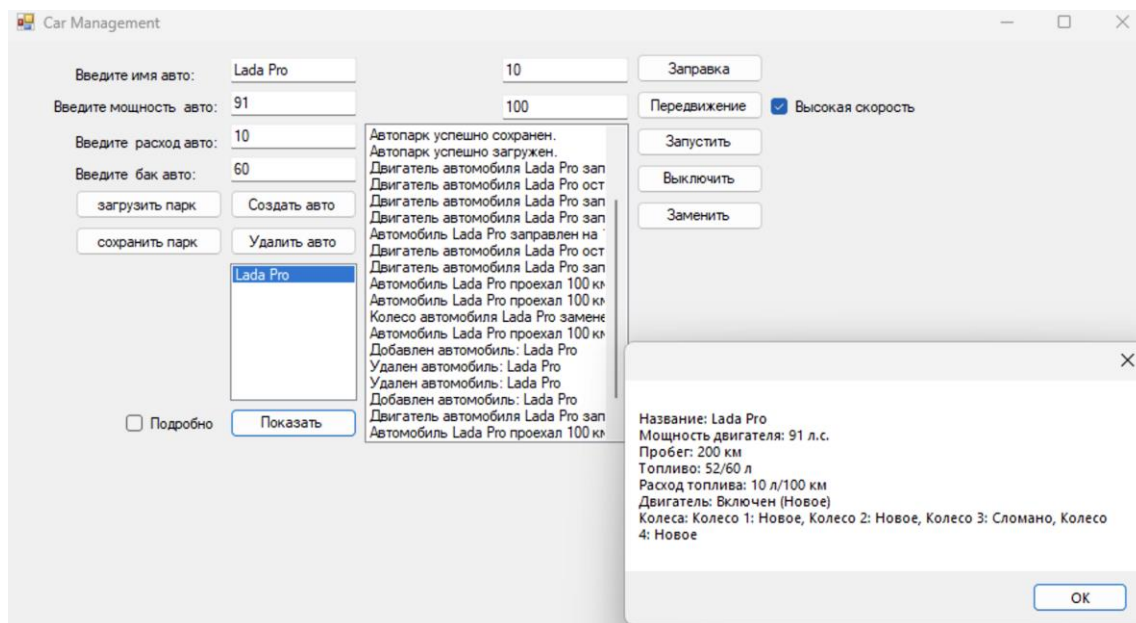


Рисунок 18 – Тест заправки

Заправка работает корректно, заправляет правильно, что дает возможность двигаться дальше.

5.5 Запуск и выключение двигателя

Чтобы продолжить движение после заправки, нужно его запустить, ведь во время заправки двигатель автоматически выключается. На рисунке 19 представлен тест заправки автомобиля.

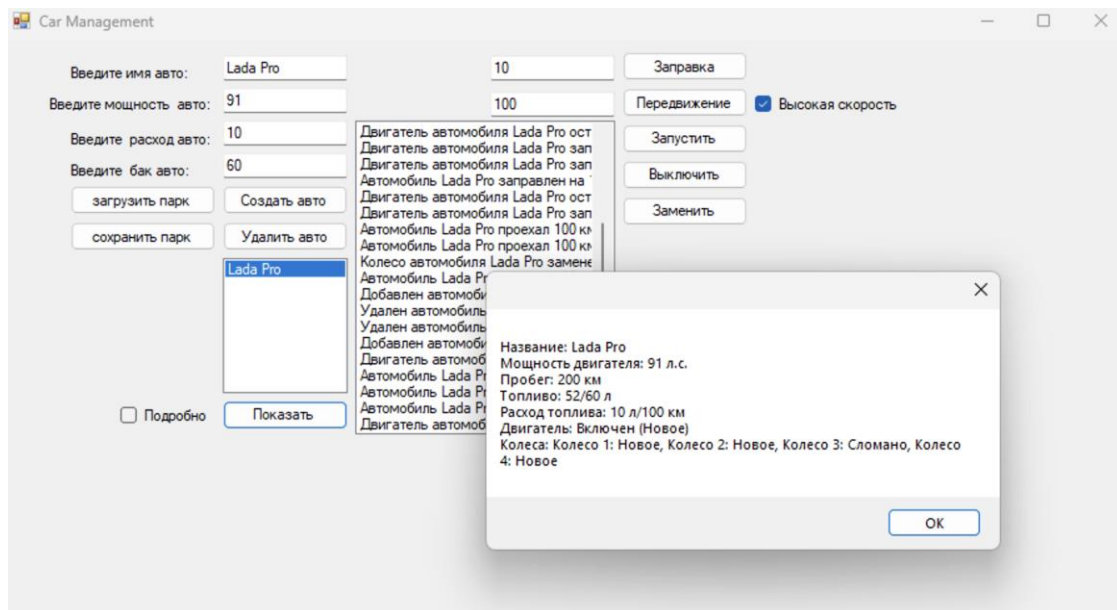


Рисунок 19 – Тест заправки автомобиля

6 Использование библиотеки в приложении

Создадим автомобиль, вписав в форму создания все необходимые параметры для экземпляра. На рисунке 20 подробно показано создание автомобиля в интерфейсе приложения.

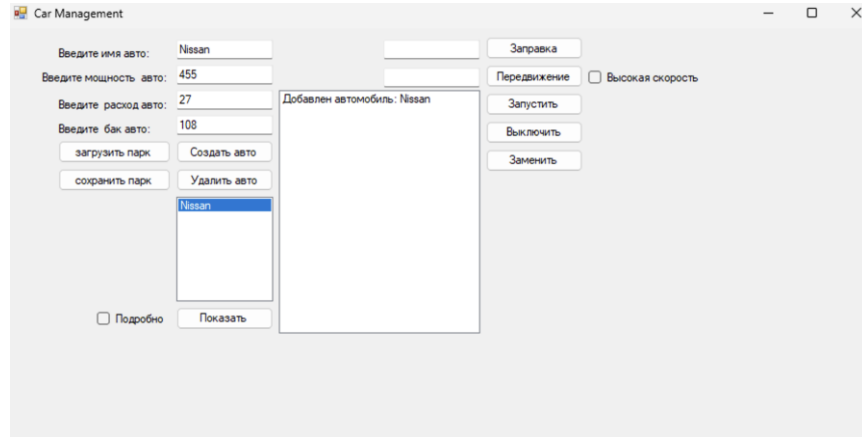


Рисунок 20 – Создание автомобиля

Первым делом мы попытаемся передвигаться на автомобиле без запущенного двигателя. При попытке движения приложение выдаст ошибку, указывая на то, что двигатель выключен и движение невозможно. Это важный момент, который демонстрирует, что система правильно обрабатывает состояние автомобиля. На рисунке 21 показано езда автомобиля без включенного двигателя.

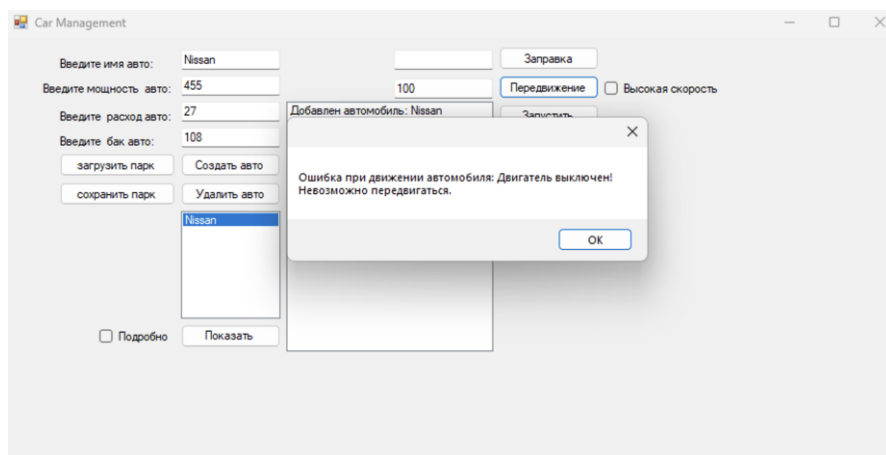


Рисунок 21 – Езда без включенного двигателя

После этого мы запустим двигатель автомобиля. Убедившись, что двигатель работает, мы сможем проехать 200 км с обычной скоростью. Приложение должно корректно обновить пробег и состояние топлива, а также

отобразить информацию о выполненном действии в списке действий. Также заменим колесо, чтобы продолжить движения, ведь после 200 км одно из четырех сломалось. На рисунке 22 показано езда автомобиля и замена колеса.

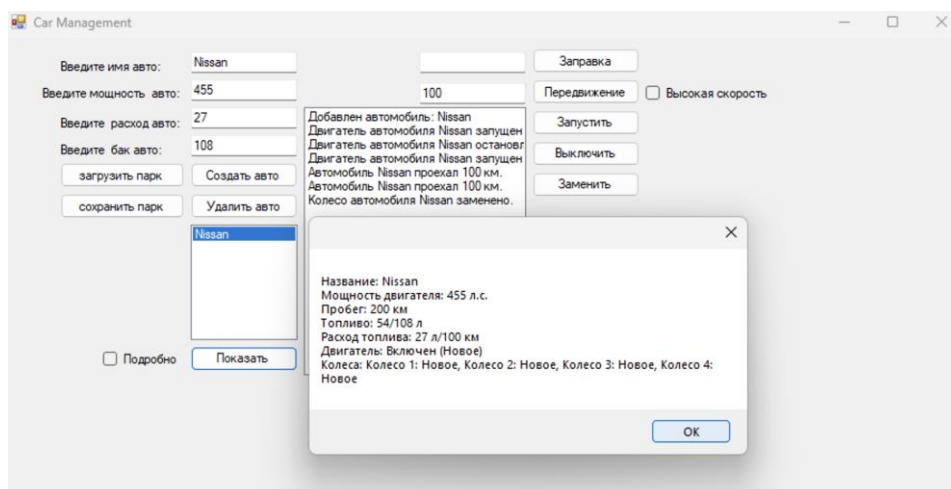


Рисунок 22 – Езда 200 км и смена колеса

Затем мы попробуем проехать 200 км на высокой скорости. В этом случае приложение должно учесть, что расход топлива снижается на 20%, и обновить данные о топливе соответственно. Мы также проверим, что пробег увеличивается, и все изменения корректно отображаются в интерфейсе. И забудем поменять колесо после движения на 200 км. На рисунке 23 показано езда автомобиля без замена колеса.

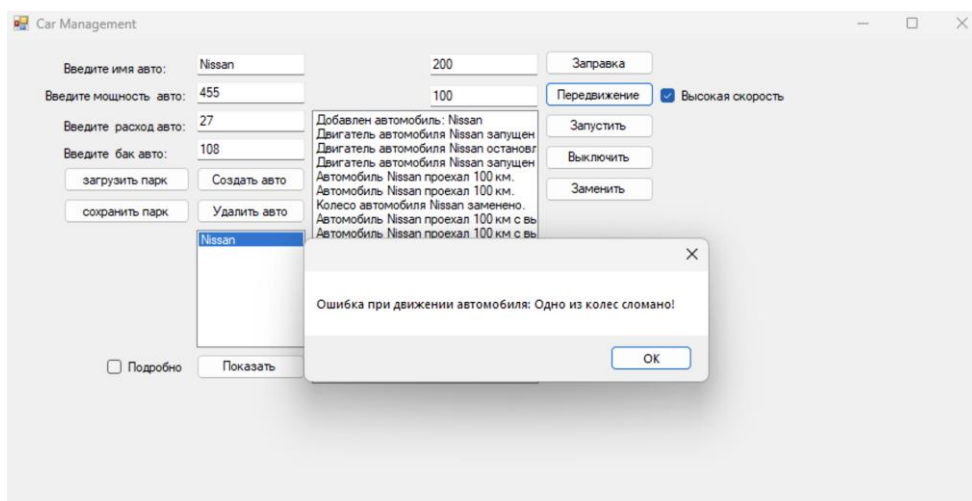


Рисунок 23 – Езда 200 км с высокой скоростью без смены колеса

После поездки мы заправим автомобиль, добавив 200 литров топлива. Приложение должно обновить состояние бака, и мы увидим, что количество топлива увеличилось, не превышая максимальную емкость бака. И проедем до

поломки двигателя. Затем мы намеренно ломаем двигатель, установив его состояние как "сломано". При попытке движения приложение снова выдаст ошибку, указывая на то, что двигатель сломан и движение невозможно. На рисунке 24 показано заправка и поломка двигателя.

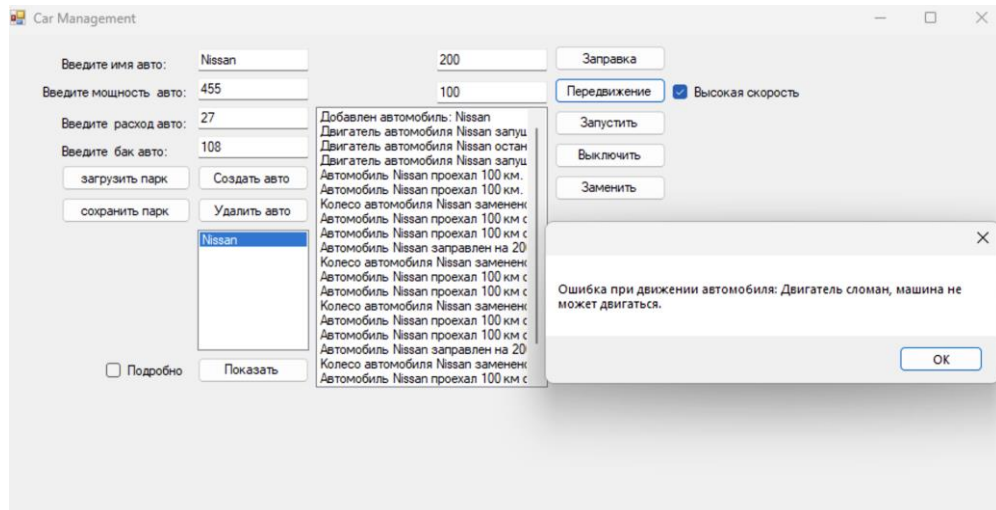


Рисунок 24 – Заправка свыше бака км и поломка двигателя

В конце концов, мы удалим наш Nissan. Приложение должно корректно обработать удаление автомобиля, и мы увидим соответствующее сообщение в списке действий. На рисунке 25 показано удаление авто.

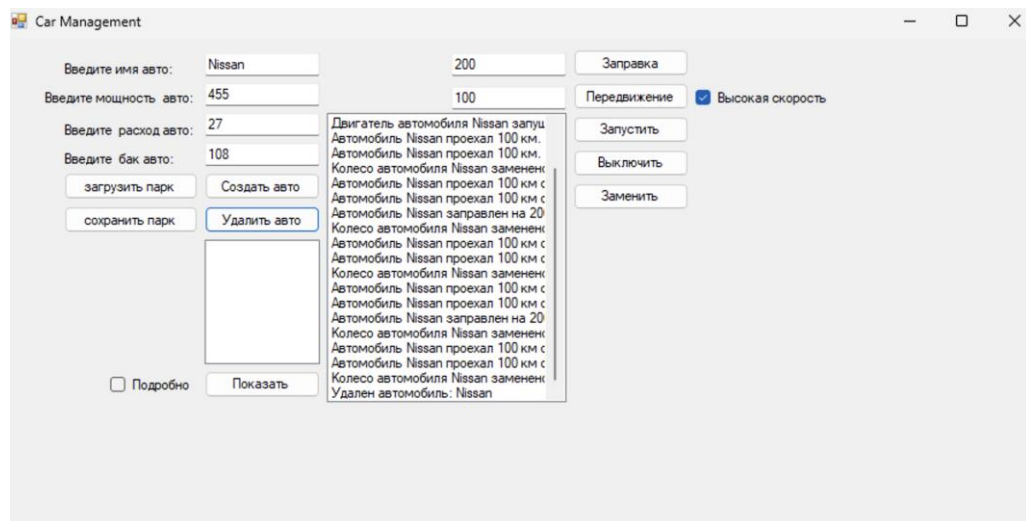


Рисунок 25 – Удаление авто

7 Руководство пользователя

Для работы с библиотекой необходим компилятор C#, встроенный в среду разработки Microsoft Visual Studio. После установки среды программирования, необходимо установить .Net 8.1 SDK. Библиотека не имеет других зависимостей и готова к использованию.

8 Заключение

Код проекта выложен и доступен на удаленном репозитории на GitHub:

<https://github.com/skv0r/car-management-app>

В результате выполнения курсового проекта была разработана библиотека классов, для создания и управления автомобилем. Классы библиотеки содержат все необходимые члены (поля, конструкторы, свойства, методы).

В данной библиотеке реализованы основные принципы объектно-ориентированного программирования: полиморфизм (представлен использованием обобщенных типов), инкапсуляция (представлен методами и свойствами), наследование (представлен реализацией интерфейсов).

Функционал библиотеки успешно протестирован. Все поставленные задачи реализованы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП, документация для учебного процесса. – URL: <https://guap.ru/regdocs/docs/uch> (дата обращения 27.11.2024)
2. Microsoft Docs. Interface (справочник по C#) – URL: <https://docs.microsoft.com/ru-ru/dotnet/csharp/language-reference/keywords/interface> (дата обращения 26.11.2024)
3. События C# (справочник по C#) – URL: <https://metanit.com/sharp/tutorial/3.14.php> (дата обращения 25.11.2024)
4. Гергель, В.П. Современные языки и технологии параллельного программирования: Учебник / В.П. Гергель. — М.: МГУ, 2016. — 408 с.
5. Программирование на C# для начинающих сост.: А.Н. Васильев, изд. «Бомбора», 2022г.
6. Head First. Изучаем C#» сост.: Э. Стилмен, Д.Грин, изд. «Питер, 2022г.

ПРИЛОЖЕНИЕ

Исходный код программы

```
using System;
using System.Linq;

namespace CarManagementApp
{
    // Класс Car представляет автомобиль с его компонентами (двигатель,
    колеса) и управлением ими
    public class Car
    {
        // Свойство Name хранит название автомобиля с обработкой лишних
        пробелов
        private string _name;
        public string Name
        {
            get => _name;
            set => _name = value?.Trim() ?? throw new
            ArgumentException("Название не может быть пустым.");
        }

        // Свойство Engine хранит объект двигателя автомобиля
        public Engine Engine { get; set; }

        // Свойство Wheels хранит массив колес автомобиля (массив из 4 колес)
        public Wheel[] Wheels { get; set; }

        // Свойство Fuel хранит текущее количество топлива в автомобиле с
        проверкой на отрицательные значения
        private int _fuel;
```

```

public int Fuel
{
    get => _fuel;
    set
    {
        if (value < 0)
            throw new ArgumentException("Количество топлива не может
быть отрицательным.");
        _fuel = value;
    }
}

// Свойство FuelTankCapacity хранит максимальный объем топливного
бака
public int FuelTankCapacity { get; private set; }

// Свойство Mileage хранит пробег автомобиля в километрах
public int Mileage { get; set; } = 0;

// Свойство FuelConsumption хранит расход топлива на 100 км
public int FuelConsumption { get; private set; }

// Статическое свойство для общего количества колес
public static int WheelCount { get; } = 4;

// Свойство Symbol проверяет регистр символа и преобразует строчные
буквы в прописные
private char _symbol;
public char Symbol
{

```

```

    get => _symbol;
    set => _symbol = char.IsLower(value) ? char.ToUpper(value) : value;
}

```

// Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность

```

private DateTime _lastServiceDate;
public DateTime LastServiceDate
{
    get => _lastServiceDate;
    set
    {
        if (value < DateTime.Now)
            throw new ArgumentException("Дата не может быть в прошлом.");
        _lastServiceDate = value;
    }
}

```

// Конструктор класса Car

// Инициализирует автомобиль с заданными параметрами: имя, мощность двигателя, расход топлива и объем бака

```

public Car(string name, int horsepower, int fuelConsumption, int
fuelTankCapacity)
{
    Name = name;

    Engine = new Engine { HorsePower = horsepower };
    Wheels = new Wheel[4]; // Инициализация массива колес
}

```

```
// Инициализация колес и подписка на событие поломки каждого
колёса
for (int i = 0; i < WheelCount; i++)
{
    Wheels[i] = new Wheel();
    Wheels[i].ComponentBroken += OnComponentBroken; // Подписка на
событие поломки колеса
}
```

```
Engine.ComponentBroken += OnComponentBroken; // Подписка на
событие поломки двигателя
```

```
FuelConsumption = fuelConsumption;
FuelTankCapacity = fuelTankCapacity;
Fuel = FuelTankCapacity; // Заполнение бака до максимума
}
```

```
// Свойство IsEngineOn проверяет, работает ли двигатель
// Возвращает true, если двигатель включен, иначе false
public bool IsEngineOn => Engine.IsRunning;
```

```
// Метод StartEngine запускает двигатель
public void StartEngine() => Engine.Start();
```

```
// Метод StopEngine выключает двигатель
public void StopEngine() => Engine.Stop();
```

```
// Обработчик события поломки компонента (колеса или двигателя)
// При поломке компонента выводится сообщение о поломке в консоль
```

```

private void OnComponentBroken(object sender, EventArgs e)
{
    if (sender is Engine)
    {
        // Сообщение о поломке двигателя
        Console.WriteLine($"{Name}: Двигатель сломан!");
    }
    else if (sender is Wheel wheel)
    {
        // Сообщение о поломке колеса
        Console.WriteLine($"{Name}: Колесо {Array.IndexOf(Wheels, wheel)
+ 1} сломано!");
    }
}

// Перегрузка метода Move: стандартное движение (обычная скорость)
// Этот метод вызывает второй метод Move с параметром isFast = false
(стандартная скорость)
public void Move(int distance)
{
    Move(distance, false); // Перемещение с обычной скоростью
}

// Перегрузка метода Move: движение с дополнительными параметрами
(например, высокая скорость)
// Если параметр isFast равен true, расход топлива снижается на 20%
(ускоренный режим)
public void Move(int distance, bool isFast = false)
{
    // Проверка, включен ли двигатель, если выключен - нельзя двигаться

```

```

    if (!IsEngineOn)
    {
        throw new InvalidOperationException("Двигатель выключен!
Невозможно передвигаться.");
    }

    // Проверка, не сломан ли двигатель
    if (Engine.Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, машина не
может двигаться.");

    // Проверка, достаточно ли топлива для движения
    if (Fuel < (distance * FuelConsumption) / 100)
        throw new InvalidOperationException("Недостаточно топлива!");

    // Проверка, не сломано ли одно из колес
    if (Wheels.Any(w => w.Condition == "Сломано"))
        throw new InvalidOperationException("Одно из колес сломано!");

    // Увеличиваем пробег
    Mileage += distance;

    // Расчет расхода топлива в зависимости от скорости
    if (isFast)
    {
        // Ускоренная скорость: расход топлива снижается на 20%
        Fuel -= (int)((distance * FuelConsumption) / 100 * 0.8); // Приведение
типа к int
    }
    else

```

```

{
    // Обычная скорость: расход топлива без изменений
    Fuel -= (distance * FuelConsumption) / 100;
}

// Поломка колеса каждые 200 км
if (Mileage % 200 == 0)
{
    Random random = new Random();
    int brokenWheelIndex = random.Next(Wheels.Length);
    Wheels[brokenWheelIndex].ChangeCondition("Сломано"); //
    Случайная поломка одного из колес
}

// Поломка двигателя каждые 1000 км
if (Mileage % 1000 == 0)
{
    Engine.ChangeCondition("Сломано"); // Поломка двигателя после
    1000 км пробега
}
}

// Метод Refuel заправляет автомобиль на указанное количество литров
// Проверяет, не превышает ли заправка емкость бака
public void Refuel(int amount)
{
    if (amount <= 0)
        throw new ArgumentException("Заправляемое количество должно
        быть положительным!");
}

```

```

        Fuel += amount;
        if (Fuel > FuelTankCapacity)
            Fuel = FuelTankCapacity; // Если заправлено больше, чем вмещает
бак, устанавливаем максимальную емкость
    }

    // Метод ReplaceBrokenWheel заменяет сломанное колесо на новое
    public void ReplaceBrokenWheel()
    {
        for (int i = 0; i < Wheels.Length; i++)
        {
            if (Wheels[i].Condition == "Сломано")
            {
                Wheels[i].ChangeCondition("Новое"); // Заменить сломанное
колесо
            }
        }
        return;
    }

    throw new InvalidOperationException("Нет сломанных колес для
замены."); // Если нет сломанных колес
}

// Метод для получения информации о машине
// Если параметр showDetailed = true, выводится подробная информация
    public virtual string GetCarInfo(bool showDetailed = false)
    {
        // Основная информация о машине
        string info = $"Название: {Name}\n" +
            $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
            $"Пробег: {Mileage} км\n" +

```

```

        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")}\n" +
        ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

```

// Если showDetailed == true, добавляется подробная информация

```

        return info;
    }
}

```

using System;

using System.Collections.Generic;

namespace CarManagementApp

{

public class CarPark

{

// Список автомобилей в автопарке

public List<Car> Cars { get; set; } = new List<Car>();

// Добавление автомобиля в автопарк

public void AddCar(Car car)

{

Cars.Add(car); // Добавляем новый объект Car в список Cars

```

    }

    // Удаление автомобиля из автопарка по имени
    public void RemoveCar(string carName)
    {
        // Ищем машину по имени в списке
        Car carToRemove = Cars.Find(c => c.Name == carName);
        if (carToRemove != null)
        {
            Cars.Remove(carToRemove); // Удаляем найденный объект Car
        }
    }

    // Получение автомобиля по имени
    public Car GetCar(string carName)
    {
        // Ищем машину по имени в списке и возвращаем её
        return Cars.Find(c => c.Name == carName);
    }
}

```

```

using System;

```

```

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и
    события для всех компонентов автомобиля
}

```

```

// Свойство Condition хранит текущее состояние компонента, по
умолчанию "Новое"
public string Condition { get; protected set; } = "Новое";

// Метод ChangeCondition для изменения состояния компонента
// Проверяет, что состояние может быть только "Новое" или "Сломано"
// При изменении состояния на "Сломано" вызывает событие поломки
компонента
public void ChangeCondition(string condition)
{
    if (condition != "Новое" && condition != "Сломано")
        throw new ArgumentException("Неверное состояние компонента");

    Condition = condition;

    // Если состояние компонента "Сломано", генерируем событие поломки
    if (Condition == "Сломано")
    {
        OnComponentBroken();
    }
}

// Защитный метод для вызова события поломки компонента
// Срабатывает, если компонент сломался
protected virtual void OnComponentBroken()
{
    ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события
ComponentBroken
}

```

```

        // Событие поломки компонента, которое оповещает об изменении
состояния на "Сломано"
        public event EventHandler ComponentBroken;
    }

using System;

// Класс Engine наследуется от Component и представляет двигатель
автомобиля
// Класс управляет состоянием двигателя (работает/не работает) и его
мощностью
public class Engine : Component
{
    // Свойство HorsePower хранит мощность двигателя в лошадиных силах
    public int HorsePower { get; set; }

    // Свойство IsRunning указывает, работает ли двигатель (true - работает,
false - не работает)
    public bool IsRunning { get; private set; }

    // Метод Start запускает двигатель, если его состояние не "Сломано"
    // Если двигатель сломан, выбрасывается исключение
    public void Start()
    {
        if (Condition == "Сломано")
            throw new InvalidOperationException("Двигатель сломан, его нельзя
запустить.");
        IsRunning = true;
    }
}

```

```

// Метод Stop останавливает двигатель, если его состояние не "Сломано"
// Если двигатель сломан, выбрасывается исключение
public void Stop()
{
    if (Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, его нельзя
остановить.");
    IsRunning = false;
}

// Переопределение метода ChangeCondition для изменения состояния
двигателя
// Изменяет состояние и вызывает событие поломки, если двигатель стал
"Сломан"
public void ChangeCondition(string condition)
{
    Condition = condition;
}
}
)

using System;

public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента
    (например, "Новое", "Сломано")
    string Condition { get; set; }
}

```

```

        // Метод для изменения состояния компонента

        // Принимает строку, которая указывает новое состояние компонента
        (например, "Сломано" или "Новое")
        void ChangeCondition(string condition);
    }

using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {
        // Конструктор для класса Info, который вызывает конструктор базового
        класса
        public Info(string name, int horsepower, int fuelConsumption, int
        fuelTankCapacity)
            : base(name, horsepower, fuelConsumption, fuelTankCapacity)
        {
        }

        // Переопределяем метод получения информации о машине
        public override string GetCarInfo(bool showDetailed = false)
        {
            string info = $"Название: {Name}\n" +
                $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
                $"Пробег: {Mileage} км\n" +
                $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
                $"Расход топлива: {FuelConsumption} л/100 км\n" +
                $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")}"
            ({Engine.Condition})\n" +

```

```
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";
```

```
        if (showDetailed)
        {
            info += $"\\nОставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
            for (int i = 0; i < Wheels.Length; i++)
            {
                info += $"\\nКолесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
            }
        }

        return info;
    }
}
```

```
namespace CarManagementApp
{
    // Интерфейс IVehicle, который определяет методы для управления транспортным средством
    public interface IVehicle
    {
        // Метод для запуска двигателя
        void StartEngine(); // Запуск двигателя

        // Метод для остановки двигателя
        void StopEngine(); // Остановка двигателя
    }
}
```

```

    }
}
using System;
using System.Windows.Forms;

namespace CarManagementApp
{
    static class Program
    {
        [STAThread]
        static void Main()
        {
            // Инициализация графического интерфейса
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1()); // Запуск главной формы
        }
    }
}
using System;

public abstract class Vehicle
{
    public string Name { get; set; } // Название транспортного средства

    // Базовый метод для получения информации о транспортном средстве.
    // Этот метод будет переопределяться в наследующих классах.
    public virtual string GetInfo(bool showDetailed)
    {
        // Строка с основной информацией о транспортном средстве

```

```

string info = $"Название: {Name}";

if (showDetailed)
{
    // Добавление подробной информации, если параметр showDetailed
    равен true
    info += GetDetailedInfo();
}

return info; // Возвращаем информацию о транспортном средстве
}

// Этот метод можно переопределить в дочерних классах для
предоставления подробной информации
protected virtual string GetDetailedInfo()
{
    // Возвращаем информацию о названии транспортного средства
    return $"Название: {Name}";
}
}

using System;

public class Wheel : IComponent
{
    public string Condition { get; set; } = "Новое"; // Состояние колеса
    public int BrokenCount { get; set; } = 0; // Счетчик поломок колеса

    // Событие поломки колеса
    public event EventHandler ComponentBroken;

```

```

// Метод для восстановления колеса
public void Repair()
{
    // Если колесо уже новое, невозможно выполнить восстановление
    if (Condition == "Новое")
    {
        throw new InvalidOperationException("Колесо уже в хорошем
состоянии.");
    }
    // Восстановление состояния колеса в "Новое"
    Condition = "Новое";
    // Генерация события о восстановлении колеса
    ComponentBroken?.Invoke(this, EventArgs.Empty);
}

// Метод для изменения состояния колеса
public void ChangeCondition(string condition)
{
    // Проверка, что состояние колеса либо "Новое", либо "Сломано"
    if (condition != "Новое" && condition != "Сломано")
        throw new ArgumentException("Неверное состояние компонента");

    // Изменение состояния колеса
    Condition = condition;

    // Если колесо сломано, увеличиваем счетчик поломок и генерируем
событие
    if (Condition == "Сломано")
    {
        BrokenCount++; // Увеличиваем счетчик поломок

```

```

        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация
события при поломке
    }
}

// Метод для получения информации о колесе
public string GetInfo(bool showDetailed)
{
    // Базовая информация о состоянии колеса
    string info = $"Колесо состояние: {Condition}";

    if (showDetailed)
    {
        // Если запрашиваются подробности, добавляем информацию о
количестве поломок
        info += $"\\nКоличество поломок: {BrokenCount}";
    }

    return info; // Возвращаем итоговую информацию
}
}

using System;
using System.IO;
using System.Linq;
using System.Windows.Forms;
using CarManagementApp;

namespace CarManagementApp
{
    public partial class Form1 : Form

```

```

{
    // Объект автопарка, который будет хранить все автомобили
    private CarPark carPark = new CarPark();

    // Конструктор формы, инициализация компонентов
    public Form1()
    {
        InitializeComponent();
    }

    // Обработчик кнопки "Создать авто"
    // При нажатии создает новый объект Car с параметрами из текстовых
    // полей, добавляет его в автопарк
    private void buttonAddCar_Click(object sender, EventArgs e)
    {
        try
        {
            string name = textBoxCarName.Text;
            int horsepower = int.Parse(textBoxHorsePower.Text);
            int fuelConsumption = int.Parse(textBoxFuelConsumption.Text);
            int fuelTankCapacity = int.Parse(textBoxFuelTankCapacity.Text);

            // Создаем объект типа Info вместо базового Car
            var car = new Info(name, horsepower, fuelConsumption,
fuelTankCapacity);

            carPark.AddCar(car);

            listBoxCars.Items.Add(name);
            listBoxActions.Items.Add($"Добавлен автомобиль: {name}");

```

```

    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при добавлении автомобиля:
{ex.Message}");
    }
}

```

```

// Обработчик кнопки "Удалить авто"
// При нажатии удаляет выбранный автомобиль из автопарка
private void buttonRemoveCar_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            // Удаляем машину из автопарка
            carPark.RemoveCar(selectedCar);

            // Обновляем интерфейс: удаляем машину из списка и выводим
            действия
            listBoxCars.Items.Remove(selectedCar);
            listBoxActions.Items.Add($"Удален автомобиль: {selectedCar}");
        }
        else
        {
            // Если машина не выбрана, выводим предупреждение

```

```

        MessageBox.Show("Выберите автомобиль для удаления.");
    }
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при удалении автомобиля:
{ex.Message}");
}
}

// Обработчик кнопки "Запустить двигатель"
// При нажатии запускает двигатель выбранного автомобиля
private void buttonStartEngine_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            // Получаем объект автомобиля из автопарка и запускаем
двигатель
            Car car = carPark.GetCar(selectedCar);
            car.StartEngine();

            // Обновляем интерфейс: выводим действия
            listBoxActions.Items.Add($"Двигатель автомобиля {selectedCar}
запущен.");
        }
    }
}

```

```

else
{
    // Если машина не выбрана, выводим предупреждение
    MessageBox.Show("Выберите автомобиль для запуска
двигателя.");
}
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при запуске двигателя: {ex.Message}");
}
}

```

```

// Обработчик кнопки "Выключить двигатель"
// При нажатии выключает двигатель выбранного автомобиля
private void buttonStopEngine_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            // Получаем объект автомобиля из автопарка и выключаем
двигатель
            Car car = carPark.GetCar(selectedCar);
            car.StopEngine();
        }
    }
}

```

// Обновляем интерфейс: выводим действия

```

        listBoxActions.Items.Add($"Двигатель автомобиля {selectedCar}
остановлен.");
    }
    else
    {
        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для остановки
двигателя.");
    }
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при остановке двигателя:
{ex.Message}");
}
}

// Обработчик кнопки "Заправка"
// При нажатии заправляет выбранный автомобиль на указанное
количество топлива
private void buttonRefuel_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            // Получаем количество топлива из текстового поля

```

```

int fuelAmount = int.Parse(textBoxFuelAmount.Text);

// Получаем объект автомобиля из автопарка и заправляем его
Car car = carPark.GetCar(selectedCar);
car.Refuel(fuelAmount);

// Обновляем интерфейс: выводим действия
listBoxActions.Items.Add($"Автомобиль {selectedCar} заправлен
на {fuelAmount} литров.");
    }
else
    {
        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для заправки.");
    }
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при заправке автомобиля:
{ex.Message}");
}
}

// Обработчик кнопки "Передвижение"
// При нажатии передвигает выбранный автомобиль на указанное
расстояние
private void buttonMove_Click(object sender, EventArgs e)
{
    try

```

```

{
    // Получаем выбранную машину из списка
    string selectedCar = listBoxCars.SelectedItem?.ToString();
    if (selectedCar != null)
    {
        // Получаем расстояние из текстового поля
        int distance = int.Parse(textBoxDistance.Text);

        // Проверка, выбрал ли пользователь ускоренный режим
движения
        bool isFast = checkBoxFastMode.Checked;

        // Вызов перегруженного метода Move с параметром для высокой
скорости
        if (isFast)
        {
            // Перемещение с высокой скоростью
            Car car = carPark.GetCar(selectedCar);
            car.Move(distance, true); // Перегруженная версия метода Move с
параметром isFast
            listBoxActions.Items.Add($"Автомобиль {selectedCar} проехал
{distance} км с высокой скоростью.");
        }
        else
        {
            // Стандартное перемещение
            Car car = carPark.GetCar(selectedCar);
            car.Move(distance); // Стандартная версия метода Move
            listBoxActions.Items.Add($"Автомобиль {selectedCar} проехал
{distance} км.");
        }
    }
}

```

```

        }
    }
    else
    {
        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для передвижения.");
    }
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при движении автомобиля:
{ex.Message}");
}
}

// Обработчик кнопки "Заменить колесо"
// При нажатии заменяет сломанное колесо выбранного автомобиля
private void buttonReplaceWheel_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            // Получаем объект автомобиля из автопарка и заменяем колесо
            Car car = carPark.GetCar(selectedCar);
            car.ReplaceBrokenWheel();
        }
    }
    catch { }
}

```

```

        // Обновляем интерфейс: выводим действия
        listBoxActions.Items.Add($"Колесо автомобиля {selectedCar}
заменено.");
    }
    else
    {
        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для замены колеса.");
    }
}
catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при замене колеса: {ex.Message}");
}
}

```

```

// Обработчик кнопки "Показать информацию"
// При нажатии отображает информацию о выбранном автомобиле
private void buttonShowInfo_Click(object sender, EventArgs e)
{
    try
    {
        // Проверяем, был ли выбран автомобиль
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            Car car = carPark.GetCar(selectedCar);

            if (car == null)

```

```

    {
        MessageBox.Show("Ошибка: Автомобиль не найден.");
        return;
    }

    // Добавляем диагностику типа объекта
    if (car is Info infoCar)
    {
        // Если это объект типа Info, используем метод Info
        bool showDetails = checkBoxShowDetails.Checked;
        string info = infoCar.GetCarInfo(showDetails);
        MessageBox.Show(info);
    }
    else
    {
        // В противном случае используем метод базового класса
        bool showDetails = checkBoxShowDetails.Checked;
        string info = car.GetCarInfo(showDetails);
        MessageBox.Show(info);
    }
}
else
{
    MessageBox.Show("Выберите автомобиль для отображения
информации.");
}
}
catch (Exception ex)
{

```

```

        MessageBox.Show($"Ошибка при отображении информации:
{ex.Message}");
    }
}

private void button1_Click(object sender, EventArgs e)
{

}

// Обработчик кнопки "Сохранить автопарк"
private void buttonSaveCarPark_Click(object sender, EventArgs e)
{
    try
    {
        // Указываем путь к файлу (можно сделать выбор пути через
        диалоговое окно)
        string filePath = "CarPark.txt";

        using (var writer = new StreamWriter(filePath))
        {
            // Сохраняем данные о каждом автомобиле в автопарке
            foreach (var car in carPark.Cars)
            {

writer.WriteLine($"{car.Name};{car.Engine.HorsePower};{car.FuelConsumption
};{car.FuelTankCapacity};{car.Fuel};{car.Mileage};{car.Engine.Condition};{strin
g.Join(",", car.Wheels.Select(w => w.Condition))}");
            }
        }
    }
}

```

```

        listBoxActions.Items.Add("Автопарк успешно сохранен.");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при сохранении автопарка:
{ex.Message}");
    }
}

// Обработчик кнопки "Загрузить автопарк"
private void buttonLoadCarPark_Click(object sender, EventArgs e)
{
    try
    {
        // Указываем путь к файлу
        string filePath = "CarPark.txt";

        if (!File.Exists(filePath))
        {
            MessageBox.Show("Файл с автопарком не найден.");
            return;
        }

        carPark.Cars.Clear(); // Очищаем текущий автопарк
        listBoxCars.Items.Clear(); // Очищаем список на форме

        using (var reader = new StreamReader(filePath))
        {
            string line;

```

```

while ((line = reader.ReadLine()) != null)
{
    // Разбираем строку файла
    var data = line.Split(';');
    string name = data[0];
    int horsepower = int.Parse(data[1]);
    int fuelConsumption = int.Parse(data[2]);
    int fuelTankCapacity = int.Parse(data[3]);
    int fuel = int.Parse(data[4]);
    int mileage = int.Parse(data[5]);
    string engineCondition = data[6];
    var wheelConditions = data[7].Split(',');

    // Создаем новый объект Car
    var car = new Car(name, horsepower, fuelConsumption,
fuelTankCapacity)
    {
        Fuel = fuel,
        Mileage = mileage
    };

    car.Engine.ChangeCondition(engineCondition);

    for (int i = 0; i < Car.WheelCount; i++)
    {
        car.Wheels[i].ChangeCondition(wheelConditions[i]);
    }

    carPark.AddCar(car); // Добавляем машину в автопарк
    listBoxCars.Items.Add(name); // Добавляем название в список

```

```

        }
    }

    listBoxActions.Items.Add("Автопарк успешно загружен.");
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка при загрузке автопарка:
{ex.Message}");
}

private void buttonLoadCarPark_Click_1(object sender, EventArgs e)
{

}
}
}

```